

DAT62-3

Feature Engineering and Model Selection

Sharpening the modeling craft

Advanced feature engineering, leakage-free selection, and model selection under business constraints.

Albert School — 22 hours

Contents

1 Feature engineering strategy	2
2 High-cardinality categorical encoding	2
3 Feature selection methods	3
4 Leakage-free feature selection	3
5 Feature store design	3
6 Automated feature generation	4
7 Model comparison	4
8 Business-constrained model selection	4
9 Written model selection report	4

1 Feature engineering strategy

This course assumes the classical ML workflow of S3–S4: linear and logistic regression, decision trees, random forests, and gradient boosting; cross-validation, feature scaling, and basic feature engineering (polynomial features, interaction terms, target encoding, date decomposition). It builds on that foundation rather than re-teaching it.

A **feature engineering strategy** for a structured dataset proceeds by identifying three things:

- **Raw variables:** the columns present in the source data — numeric, categorical, temporal, or identifier.
- **Candidate transformations:** operations that turn a raw variable into a feature more useful to a model — scaling, binning, logarithms and other nonlinear transforms, date decomposition, aggregation, and the categorical encodings below.
- **Interaction terms:** features built from two or more raw variables together (products, ratios, differences, or grouped aggregates), capturing effects that no single variable expresses on its own.

The strategy is the documented list of which raw variables to use, which transformations to apply to each, and which interactions to construct, decided before modeling.

2 High-cardinality categorical encoding

A **high-cardinality** categorical variable has many distinct levels (e.g. postal code, product ID, user ID). One-hot encoding such a variable produces too many columns to be practical. Three encodings handle high cardinality, each carrying a distinct **leakage** risk — the risk that information from the target or from outside the training fold contaminates a feature.

- **Target encoding** replaces each category with a statistic of the target computed over the rows in that category (typically the mean target). **Leakage risk:** the encoding is computed from the target, so a row’s own target can leak into its own feature. It must be computed from training data only, with smoothing or out-of-fold computation, never using the row being encoded.
- **Frequency encoding** replaces each category with its count or relative frequency. **Leakage risk:** lower, since it does not use the target, but the frequencies must still be computed on the training set only — using counts from the full dataset leaks information about the validation and test rows.
- **Embedding-based encoding** maps each category to a learned dense vector (an **embedding**), trained jointly with a model. **Leakage risk:** the embedding is fitted using the target through the model’s loss, so it must be learned on training data only; reusing embeddings fitted on validation or test rows leaks the target.

In each case the encoder is **fitted on training data and applied to held-out data**, the same discipline as any other learned transformation.

3 Feature selection methods

Feature selection chooses a subset of features to keep. Three families:

- **Filter methods** rank features by a statistic computed independently of any model, then keep the top ones. Examples: **correlation** with the target, and **mutual information** (a measure of statistical dependence that, unlike correlation, captures nonlinear relationships). Fast, but they ignore feature interactions and the model that will be used.
- **Wrapper methods** search over feature subsets by repeatedly training a model and scoring it. **Recursive feature elimination (RFE)** fits a model, removes the least important feature, and repeats until the desired number remains. Accurate for the chosen model but computationally expensive.
- **Embedded methods** perform selection as part of model training. **Lasso** (L1 regularization) drives some coefficients to exactly zero, dropping those features; **tree importance** from a tree-based model ranks features and lets low-importance ones be discarded. Cheaper than wrappers because selection and fitting are one step.

4 Leakage-free feature selection

Selecting features on the **full training set** and then running cross-validation on the same data biases the performance estimate **optimistically**: the selection has already seen the validation rows of each fold, so those rows are no longer genuinely held out. The reported score is too high and does not generalize.

The fix is to put feature selection **inside the cross-validation loop**. For each fold:

1. split into training and validation parts;
2. perform feature selection using the **training part only**;
3. apply the resulting subset to the validation part and score.

The selection step is repeated independently in every fold, so the validation rows of each fold are never used to choose features. In scikit-learn this is achieved by placing the selector and the model in a single **Pipeline** and cross-validating the pipeline, so every fold re-fits the selector on its own training data.

5 Feature store design

A **feature store** is the documented catalogue of a project's features. Constructing one means recording, for each feature:

- its **definition** — what the feature represents;
- its **source** — the raw variable(s) and dataset it derives from;
- its **transformation logic** — the exact computation that produces it from the source;
- its **known limitations** — missing-value behavior, staleness, leakage caveats, or populations where it is unreliable.

The feature store makes features reusable across models and projects and makes each feature's provenance auditable.

6 Automated feature generation

Automated feature generation tools (e.g. tools that build many candidate features by systematically combining and transforming raw variables) produce large numbers of features without manual design. The required skill is to **critically evaluate** the generated features rather than accepting them blindly: check each for predictive value, redundancy with existing features, leakage, and interpretability, and keep only those that survive scrutiny. Automated generation expands the candidate pool; it does not replace judgment about what belongs in the model.

7 Model comparison

A rigorous comparison evaluates **at least four algorithm families** on the **same engineered dataset**, so that differences reflect the algorithms and not differing preprocessing. The families span the supervised toolkit of S3–S4 — for example linear/logistic models, decision trees, random forests, and gradient boosting. Each model is trained and evaluated under the same cross-validation protocol and the same metric, chosen from the regression and classification metrics of the prerequisites according to the business context. The comparison produces a structured table of performance across families.

8 Business-constrained model selection

The best test metric is not automatically the right model. A **model selection protocol** must also account for **business constraints**:

- **Inference latency**: how fast a prediction must be served. A slower model may be unusable in a real-time setting regardless of accuracy.
- **Interpretability requirements**: whether decisions must be explainable (for regulation, trust, or debugging). This can favor a simpler model over a more accurate opaque one.
- **Retraining cost**: how expensive it is to retrain as data drifts. A model that is costly or slow to retrain may be impractical where frequent retraining is needed.

The protocol weighs technical performance against these constraints rather than ranking models on a metric alone. A model a student cannot justify under such constraints has no business being deployed.

9 Written model selection report

The course's deliverable is a **written model selection report** justifying the final choice. It must address **both**:

- **technical performance**: the comparison across the four-plus algorithm families on the engineered dataset, under the chosen metric and validation protocol;
- **non-technical constraints**: inference latency, interpretability, and retraining cost.

The report states the final model and argues, in writing, why it is the right choice given both the measured performance and the business constraints.